

CSE 715: Computer Graphics

Octrees, Quadtrees, BSP-Trees & Fractal-Geometry Methods

Syllabus-Mandated Coverage — “the 5 no-resource topics”

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

Source: *Computer Graphics C Version* (Hearn & Baker), Chapter 10 — §10-16 (Octrees, incl. Quadtrees), §10-17 (BSP Trees), §10-18 (Fractal-Geometry Methods)

Why this document looks different from the other chapter solutions: this topic has **0/5 hits** in the 2020–2024 past papers — there is no PYQ history to analyze. `Syllabus.txt` explicitly carves out an exception: “*from the book: Computer Graphics C version, only the following topics: 1. Octrees (how it works) page 359, Quadtree, BST-Tree 10-17, Fractal Geometry 10-18, B-spline Curve, types of Shading.*” (B-spline and types-of-Shading are already covered in [\[\[wiki/bezier-bspline-hermite—Ch9\]\]](#) and [\[\[wiki/color-shading-models—Ch11\]\]](#) respectively — this document covers the remaining, previously-uncovered piece: Octrees/BSP-Trees/Fractal Geometry.) Given the exam is tomorrow and this is the **lowest-yield item in the entire 51-topic tracker**, treat this as a stretch-goal skim only — every other TIER1/TIER2 topic and, above all, timed past-paper practice come first.

Contents

1	Octrees (§10-16)	2
2	BSP Trees (§10-17)	2
3	Fractal-Geometry Methods (§10-18)	3
3.1	Motivation and Core Definition	3
3.2	Fractal-Generation Procedure	3
3.3	Classification of Fractals	3
3.4	Fractal Dimension	4
4	Quick Summary	5

Octrees (§10-16)

An **octree** is a hierarchical tree structure used to represent solid 3D objects by recursively subdividing a bounding region of space into **octants** (eight equal sub-cubes). Each node has **8 data elements**, one per octant; the smallest unit of 3D space is a **voxel** (volume element).

- If all voxels in an octant are the same type (**homogeneous**), that octant's data element simply stores the type value (a leaf).
- If an octant is **heterogeneous**, it is subdivided further into 8 sub-octants, and the data element instead stores a pointer to the next tree node.
- Subdivision continues until every octant is homogeneous or a resolution limit is reached. Each node has 0 to 8 immediate descendants.
- **Quadtree** is the 2D analogue: a region (usually square) is divided into **4 quadrants** per node instead of 8 octants; used for 2D image/region encoding and as a stepping stone for explaining octrees.

How it works (generation): define a bounding box around the object using its min/max coordinates; test the box octant-by-octant (from any input representation: polygon mesh, curved patches, or CSG solids); homogeneous octants become leaves, heterogeneous ones recurse. **Uses:** set operations (union/intersection/difference) on two octree objects by scanning/comparing octants; 3D rotations by transforming occupied octants; visible-surface identification by searching octants front-to-back (first hit = visible) and transferring the result to a quadtree for display; medical imaging and other applications needing object cross-sections; reduces storage via spatial coherence (large homogeneous regions collapse to one node).

BSP Trees (§10-17)

A **binary space-partitioning (BSP) tree** subdivides a scene into **two** partitions at each step (vs. octrees' fixed eight), using a cutting **plane that can be at any position and orientation** — unlike an octree, whose three cutting planes at each step are always mutually perpendicular and aligned with the coordinate axes.

- **Adaptive:** since the cutting plane's position/orientation can be chosen to fit the actual spatial distribution of objects, a BSP tree can be **shallower** than the equivalent octree for the same scene, reducing tree-search time.
- **Uses:** identifying visible surfaces, and space-partitioning for **ray-tracing algorithms** (accelerating ray/object intersection tests by quickly discarding whole partitions a ray cannot reach) — a direct link to [\[\[wiki/ray-tracing—Ch12's\]\]](#) spatial-subdivision efficiency technique.

Fractal-Geometry Methods (§10-18)

Motivation and Core Definition

Euclidean-geometry methods (equations) describe manufactured objects with smooth surfaces/regular shapes well, but cannot realistically model **natural** objects (mountains, clouds, coastlines) that have irregular, fragmented features. **Fractal-geometry methods** instead describe objects **procedurally** (a repeated transformation/generation procedure) rather than with equations. **A fractal object has two characteristics:**

1. **Infinite detail at every point** — zooming into a fractal (e.g. a mountain outline) keeps revealing new jagged detail, unlike a Euclidean shape which eventually looks smooth on zooming in.
2. **Self-similarity** between the object's parts and its overall features — the exact form of self-similarity depends on which fractal representation is chosen.

Since true infinite detail can't be displayed, graphics implementations apply the generating transformation a **finite** number of times (enough to exceed pixel resolution).

Fractal-Generation Procedure

Given an initial point (or primitive) P_0 , repeatedly apply a transformation function F :

$$P_1 = F(P_0), \quad P_2 = F(P_1), \quad P_3 = F(P_2), \quad \dots$$

F may be geometric (scaling/translation/rotation), a nonlinear coordinate transform, deterministic, or randomized at each iteration. More iterations $\Rightarrow$ more detail, up to the display's pixel resolution.

Classification of Fractals

Class	Defining property
Self-similar	Parts are scaled-down copies of the whole, using a single scaling factor s for all subparts (or different factors per subpart)
Statistically self-similar	Self-similar, plus random variation applied to each scaled-down subpart (models trees, shrubs, plants)
Self-affine	Parts scaled with <i>different</i> factors s_x, s_y, s_z per coordinate direction
Statistically self-affine	Self-affine + random variation (models terrain, water, clouds)
Invariant fractal sets	Formed with nonlinear transformations: self-squaring (e.g. the Mandelbrot set, from squaring functions in complex space) and self-inverse (from inversion procedures)

Fractal Dimension

The **fractal dimension** D (a.k.a. *fractional dimension* — the source of the name “fractal”) measures the roughness/fragmentation of an object; more jagged objects have larger D . For a self-similar fractal built with a single scaling factor s , subdividing into n subparts satisfies $ns^D = 1$, giving

$$D = \frac{\ln n}{\ln(1/s)}$$

For different scaling factors s_k per subpart ($k = 1, \dots, n$), D solves the implicit relation $\sum_{k=1}^n s_k^D = 1$. **Worked examples from the text:**

- **Koch curve/snowflake:** each segment $\rightarrow$ 4 segments, scaling factor $s = 1/3$:
 $D = \frac{\ln 4}{\ln 3} \approx 1.2619$.
- **Fractal tetrahedron surface:** each face $\rightarrow$ 6 smaller faces, scaling factor $s = 1/2$: $D = \frac{\ln 6}{\ln 2} \approx 2.58496$ (a fairly fragmented surface).

Dimension ranges (Euclidean dimension is always exceeded): a fractal *curve* has $D > 1$ ($D = 2$: a Peano curve completely fills a 2D region; $2 < D < 3$: self-intersecting, can cover an infinite area). A fractal *surface* has $2 < D \leq 3$ ($D = 3$: fills a volume; $D > 3$: overlapping coverage). A fractal *solid* has $3 < D \leq 4$. **Estimating D for irregular/non-simple shapes:** *box-covering methods* — cover the object with n small boxes (squares/cubes) of a given scaling factor and use n, s in the formula above; the more general version is the **Hausdorff–Besicovitch dimension**.

Illustrative Exercise — Fractal Dimension of a Custom Generator

Illustrative exercise (not from a past paper), same style as the worked Koch-curve example, to confirm the formula $D = \ln n / \ln(1/s)$.

Given a self-similar generator that replaces each straight segment with **5** equal-length segments, each scaled by $s = 1/5$, find the fractal dimension.

Using $D = \frac{\ln n}{\ln(1/s)}$ with $n = 5$, $s = 1/5$: $D = \frac{\ln 5}{\ln 5} = 1$. This makes sense: replacing 1 segment with 5 collinear-scale segments covering the same total length end-to-end produces no extra jaggedness beyond a straight line (unlike the Koch curve's bump, which pushes 2 of the 4 sub-segments off the original line) — a sanity check that $D = 1$ corresponds to an (effectively) non-fractal, ordinary Euclidean line.

Quick Summary

Concept	One-line memory hook
Octree	3D space split into 8 octants/node, axis-aligned; homogeneous octants are leaves
Quadtree	2D analogue of octree — 4 quadrants/node
BSP tree	2 partitions/step, cutting plane can be <i>any</i> position/orientation — adaptive, shallower than octree; used in ray-tracing space subdivision
Fractal object	Infinite detail + self-similarity; built by repeating a transformation F
Fractal dimension	$D = \ln n / \ln(1/s)$ — always exceeds the Euclidean dimension; higher D = more jagged/fragmented

Exam-day reality check: this topic has never appeared in 5 years of past papers and is the lowest-yield item in the entire tracker. If time is short, this is the first thing to skip back from — Z-Buffer, Cohen-Sutherland, and the other TIER1/TIER2 topics (and, most of all, timed past-paper practice) come first.